第 23 章：并发上限、Queue、背压与共享状态

本章最重要的一句话是：**并发程序不仅要能同时做事，还要明确最多同时做多少、来不及处理的工作放在哪里，以及失败时由谁收尾。**

一次 Agent 请求可能要查询几十条记录、调用多个 MCP tool，或向同一个模型服务发出大量 HTTP 请求。把所有 coroutine 一次性变成 Task，确实获得了并发，却没有自动解决下面的问题：

- 下游最多能承受多少个尚未结束的请求？
- 输入持续到达时，内存里最多积压多少工作？
- 生产者比消费者快时，是等待、丢弃还是拒绝？
- worker、生产者或父请求失败时，其他 Task 怎样退出？
- 多个 Task 修改同一份状态时，单线程事件循环是否足以保证安全？

本章给出两种常用结构：小批量任务使用 `TaskGroup + Semaphore`，大量或流式输入使用固定 worker + 有界 `Queue`。两者解决的是不同层次的问题，不能只用一个“并发数”概括。

学完后，你应当能够：

- 区分在途并发、请求速率、排队容量和连接池容量；
- 用 `async with Semaphore` 管理许可，并解释异常和取消时为何不会泄漏；
- 解释为什么 Semaphore 限制了运行数量，却没有限制已经创建的 Task 数；
- 用 `Queue(maxsize)` 把下游压力传回生产者；
- 正确配对 `put()`、`get()`、`task_done()` 和 `join()`；
- 为正常结束、程序失败和用户取消设计不同的 Queue 关闭路径；
- 识别跨 `await` 的共享状态竞争，并在 Lock 与改变所有权之间作出选择。

23.1 先建立 Java → Python 心智模型

可以把本章看成 Java 中“并发许可、阻塞队列、固定 worker 和结构化生命周期”的 asyncio 版本，但不能把 Task 当成线程，也不能照搬线程池的关闭方式。

Java 中熟悉的概念	Python 中可先类比的概念	不能忽略的边界
`Semaphore`	`asyncio.Semaphore`	前者常让线程等待；后者在 `await` 处暂停 Task，不占住事件循环线程，而且不是线程安全原语

`ArrayBlockingQueue`	`asyncio.Queue(maxsize=n)`	满队列时 `await put()` 暂停生产 Task，不是阻塞 OS 线程；Queue 也不负责创建 worker
固定大小 `ExecutorService`	固定数量的长期 worker Task	worker 是 coroutine Task，不是线程；worker 中调用阻塞 API 仍会卡住整个事件循环
structured-concurrency scope	`asyncio.TaskGroup`	TaskGroup 管所有权、等待和失败传播，不提供并发上限；也不返回类似 `allOf` 的结果列表
`CountDownLatch` / 阶段通知	`asyncio.Event`	Event 没有计数，只保存“未设置/已设置”状态；`set()` 会放行所有等待者，不提供互斥
`ReentrantLock` / `synchronized`	`asyncio.Lock`	asyncio Lock 用于 Task 间互斥，不可用于线程同步，也不要假定它可重入

还要特别区分 `Queue.shutdown()` 与 `ExecutorService.shutdown()`：前者关闭的是 Queue 的入队和取队协议，不会替你拥有、取消或等待 worker。worker 的生命周期仍应交给 `TaskGroup`。

本章所有 asyncio 同步原语都只服务于同一个事件循环中的 Task。若共享数据同时被线程访问，应使用线程同步机制或重新划分所有权，不能拿 `asyncio.Lock` 保护线程。

23.2 “最多 3 个”到底限制了什么

工程讨论中只说“限流为 3”是不够的，必须先说明单位。

限制	回答的问题	常用机制	达到上限后的行为
在途并发数	此刻有多少次操作已经开始但尚未结束	`Semaphore(3)` 或 3 个串行 worker	新调用等待空闲槽位
请求速率	一秒或一分钟内允许开始多少次请求	令牌桶、滑动窗口、服务端配额策略	等待下一时间窗口或明确拒绝
排队容量	有多少项尚未被 worker 领取	`Queue(maxsize=4)`	`await put()` 等待空位，形成背压
连接池容量	客户端最多建立或借出多少连接	HTTP/数据库客户端的 pool 配置	等待连接、复用连接或按客户端规则失败

Semaphore 不是 QPS 限流器

Semaphore 不是 QPS 限流器

Semaphore 的计数器没有时间概念。并发上限为 3 时：

- 若每个请求 10 毫秒结束，一秒内仍可能开始很多请求；
- 若每个请求 10 秒才结束，一秒内也可能一个都完成不了。

所以"同时最多 3 个"不能推出"每秒最多 3 次"。真正的速率策略还要定义时间窗口、突发容量、等待还是拒绝，以及服务端返回限流响应时如何处理。重试、退避和 `Retry-After` 会在第 25 章结合真实 HTTP 继续讲解。

应用并发上限不等于连接池容量

Semaphore 通常保护一段业务调用，连接池管理实际连接及其复用。假设业务许可为 10，而连接池只能借出 4 个合适的连接，已经获得业务许可的部分 Task 仍会在客户端内部等连接。反过来，连接池允许 20 个连接，也不能让 `Semaphore(3)` 保护的代码超过 3 个在途调用。

两者通常要一起配置，但数值不必相等。连接池还可能按主机、HTTP 版本或流分别计数，应以具体客户端的规则为准。

23.3 Semaphore 管的是许可，不是 Task

`asyncio.Semaphore(3)` 内部维护一个从 3 开始的计数器：

1. `acquire()` 在计数大于 0 时把它减 1，然后返回；
2. 计数为 0 时，调用方停在 `await acquire()`；
3. `release()` 把计数加 1，并可能唤醒一个等待者。

推荐把许可写成异步上下文管理器：

```
import asyncio


async def fetch_one(record_id: int) -> int:
    await asyncio.sleep(0.1)  # 代表异步 I/O
    return record_id * 2


async def limited_fetch(record_id: int, limit: asyncio.Semaphore) -> int:
    async with limit:
```

lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-4ac7ee220d97d8ab8b8febd0c61.html

... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

推荐把许可写成异步上下文管理器：

```
import asyncio


async def fetch_one(record_id: int) -> int:
    await asyncio.sleep(0.1)  # 代表异步 I/O
    return record_id * 2


async def limited_fetch(record_id: int, limit: asyncio.Semaphore) -> int:
    async with limit:,
        return await fetch_one(record_id)
```

`async with limit` 在语义上等价于：

```
await limit.acquire()
try:
    result = await fetch_one(record_id)
finally:
    limit.release()
```

`finally` 是关键。需要分别看两个取消时刻：

- Task 在等待许可时被取消： `acquire()` 没有成功返回，也没有进入上下文体，因此不应释放许可；
- Task 已持有许可时被取消：退出上下文时仍会执行释放逻辑，已经取得的许可会被归还。

不要写成 `await acquire(); await work(); release()` 。一旦 `work()` 抛异常或收到取消，最后一行就会被跳过，许可数量永久减少。若特殊控制流必须手动获取，成功 `acquire()` 后要立刻建立 `try/finally` 。

普通 `Semaphore` 允许误调用过多次 `release()` 。 `BoundedSemaphore` 会在计数超过初始值时抛 `ValueError` ，适合尽早发现不成对释放，但它不能替代正确的上下文管理。

许可应该包住真正稀缺的资源

```
request = build_request(record)  # 本地准备，不占网络槽位

async with request_limit:
    response = await client.send(request)
```

7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-4ac7ee220d97d8ab8b8febd0c61.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

许可应该包住真正稀缺的资源

```
request = build_request(record)  # 本地准备，不占网络槽位

async with request_limit:
    response = await client.send(request)
    body = await response.read()  # 若连接仍被占用，通常也留在许可内

return parse_body(body)  # 本地解析是否移出，要看真正的瓶颈
```

许可取得得太早，会让无关工作占住稀缺槽位；释放得太早，又可能让实际受限资源越过上限。真实边界要结合客户端何时归还连接、响应体是否流式读取来确定。

不要读取 Semaphore 的私有计数器来实现业务判断。是否能够进入应由 `await acquire()` 决定；指标应在受保护区域的入口和 `finally` 中单独记录。

一万个等待者仍然是一万个 Task

```
limit = asyncio.Semaphore(3)

async with asyncio.TaskGroup() as group:
    tasks = [group.create_task(limited_fetch(record_id, limit)) for record_id in range(10_000)]
```

这段代码会立即创建 10,000 个 Task。Semaphore 只让其中最多 3 个进入受保护区域，其余 Task 仍然持有 Task 对象、coroutine 状态、上下文和异常传播信息。

可以把三个组件的职责记成三句话：

- `TaskGroup` 回答“这些 Task 归谁管，失败时怎样联动”；
- `Semaphore` 回答“多少 Task 可以进入这段代码”；
- `Queue(maxsize)` 回答“尚未开始的工作最多积压多少项”。

23.4 两种并发结构应该怎样选择

已知的小批量：TaskGroup + Semaphore

小批量结构适合一次请求中的少量独立工作，例如并发调用 8 个 tool：

已知的小批量：TaskGroup + Semaphore

小批量结构适合一次请求中的少量独立工作，例如并发调用 8 个 tool：

```
async def fetch_batch(record_ids: list[int]) -> list[int]:
    limit = asyncio.Semaphore(3)

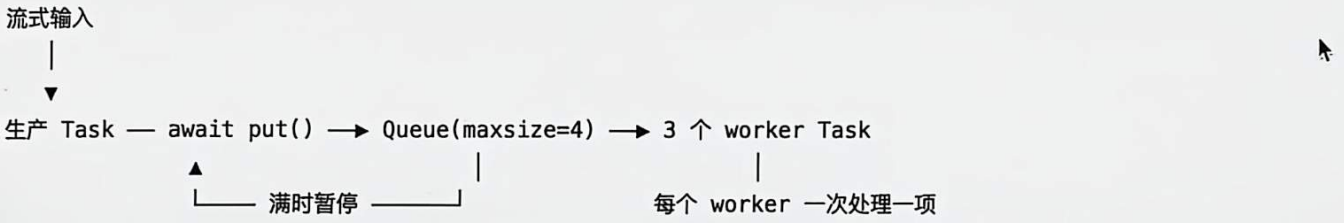
    async with asyncio.TaskGroup() as group:
        tasks = [group.create_task(limited_fetch(record_id, limit)) for record_id in record_ids]

    return [task.result() for task in tasks]
```

成功退出 TaskGroup 后，每个 Task 都已经结束，才可以同步读取 `result()` 。这种结构直观、结果与输入容易对应，但 Task 数仍随输入项数 `N` 增长。

大量或流式输入：有界 Queue + 固定 worker

输入很多、持续流入或总量未知时，不要预先为每项工作创建一个 Task。让一个生产者把普通数据项放进 Queue，再由固定数量的长期 worker 取走：



维度	小批量 + Semaphore	有界 Queue + 固定 worker
长期 Task 数	随输入 `N` 增长	生产/协调 Task 加固定 `W` 个 worker
等待工作的位置	已创建 Task 的 coroutine 状态	最多 `maxsize` 个 Queue 项
在途上限	Semaphore 的许可数	通常是 worker 数 `W`
结果关联	每项一个 Task，直接保存句柄	给数据带 ID，父任务最后按业务规则整理
适用输入	已知且不大	大量、持续或开始时未知

“worker 数就是并发数”有一个前提：每个 worker 一次只处理一项，而且内部不再创建额外并发

“worker 数就是并发数”有一个前提：每个 worker 一次只处理一项，而且内部不再创建额外并发 Task。若多个管道共享同一个下游，或 worker 内部还会扇出请求，仍可能需要一个跨管道共享的 Semaphore。

Queue 方案不是总会更快。对于 5～20 项简单工作，一项一个 Task 往往更容易阅读；Queue 的核心价值是把 Task 数和排队容量变成清楚的资源边界。

23.5 有界 Queue 怎样产生背压

创建一个只允许 4 项等待领取的 Queue：

```
queue: asyncio.Queue[Record] = asyncio.Queue(maxsize=4)

await queue.put(record)  # 满时暂停生产 Task
record = await queue.get()  # 空时暂停消费 Task
```

`maxsize=4` 只计算**已经入队但尚未被 `get()`、领取**的项目。三个 worker 已经领取并正在处理的三项不在这个数字里。

当 3 个 worker 都在处理慢请求，Queue 中又排了 4 项时，下一次 `await queue.put(record)` 无法完成。生产者让出控制权，直到某个 worker `get()` 一项腾出位置。这就是背压：下游的处理能力通过等待直接传回上游，没有轮询，也没有默认丢弃。

几个容易误判的细节：

- 默认 `maxsize=0` 表示无界，不是容量为 0；
- `put_nowait()` 在满队列上抛 `QueueFull`，它表达“立刻放入或失败”，不会形成等待式背压；
- `qsize()` 是当前准确数量，但跨过下一个 `await` 后就可能变化，不能用快照预订未来槽位；
- `asyncio.Queue[Record]` 的 `Record` 是静态类型信息，运行时不会自动校验入队对象；
- Queue 方法没有 `timeout=` 参数，需要时应在外层使用 `asyncio.timeout()` 或 `wait_for()`。

`maxsize` 只限制 Queue 自己

若 Queue 容量为 4、worker 数为 3，可以说这个管道最多有 4 项排队、3 项在途。但不能说整个程序最多保存 7 条数据，因为：

- 若输入早已构造成百万项 `list`，它们仍然全在内存中；
- 若所有结果最终收集成一个大列表，结果内存仍随输入增长；

EADME × | readme × | README × | README × | README × | README × | README × | README × | README × | README × | README ×
z72502x6dg8fpjj0sff40000gp/T/crossnote-f-4ac7ee220d97d8ab8b8febd0c61.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

`maxsize` 只限制 Queue 自己

若 Queue 容量为 4、worker 数为 3，可以说这个管道最多有 4 项排队、3 项在途。但不能说整个程序最多保存 7 条数据，因为：

- 若输入早已构造成百万项 `list`，它们仍然全在内存中；
- 若所有结果最终收集成一个大列表，结果内存仍随输入增长；
- 客户端、解析器、日志和远端 SDK 可能各有缓冲；
- 当前响应、worker Task 和其他管道也会占用资源。

真正有界的流要求输入逐项产生，结果也逐项写入下游。只在中间放一个有界 Queue，不能自动把两端已经全量物化的数据变成流。

23.6 Queue 同时有容量账本和生命周期

Java 工程师最容易把 `get()` 理解成“任务完成”。对 `asyncio.Queue` 来说，领取和完成是两件事。Queue 维护一个未完成工作计数：

```
put(item) 成功       未完成计数 +1
get() 成功           只转移项目，不修改未完成计数
task_done()            未完成计数 -1
未完成计数变为 0     join() 可以返回
```

worker 的最小正确骨架是：

```python
async def worker(queue: asyncio.Queue[Record]) -> None:
    while True:
        try:
            record = await queue.get()
        except asyncio.QueueShutDown:
            return

        try:
            await process(record)
        finally:
            queue.task_done()
```

`task_done()` 必须在一次成功 `get()` 后恰好调用一次：

`task_done()` 必须在一次成功 `get()` 后恰好调用一次：

- 漏掉一次，未完成计数无法归零，`join()` 可能永久等待；
- 多调用一次，Queue 会抛 `ValueError`；
- `get()` 因关闭或取消而没有返回项目时，不能调用 `task_done()`；
- 处理成功、可恢复业务失败、程序异常或取消，都不能破坏已经领取项目的本地记账。

`task_done()` 的名字容易让人误以为它在证明业务成功。它实际上只是消费者对 Queue 账本的确认。
业务成功、失败、重试或补偿必须由结果对象和异常策略另外表达。

`join()` 只等待账本归零

`await queue.join()` 能说明每次 `put()` 都收到了相应的 `task_done()`；它不能单独证明：

- 每项业务都成功；
- worker 已退出；
- 远端事务或副作用已经提交；
- 消费者没有过早调用 `task_done()`；
- 紧急关闭时被丢弃的项目已经执行。

因此，“完成 Queue 工作”和“结束 worker 生命周期”必须分别设计。

23.7 正常关闭、失败关闭和取消是三条路径

本项目要求 Python `>=3.14`，因此可以使用 Python 3.13 加入的 `Queue.shutdown()`。

正常结束：停止接收，排空，再退出

生产者成功放完全部输入后，正常协议是：

```
for record in records:
    await queue.put(record)

queue.shutdown()  # immediate=False: 不再接收新项目
await queue.join()  # 等已入队项目完成记账
```

执行顺序是：

1. `shutdown()` 让后续和正在等待的 `put()` 失败，不再允许 Queue 增长；

执行顺序是：

1. `shutdown()` 让后续和正在等待的 `put()` 失败，不再允许 Queue 增长；
2. 已经入队的项目仍可被 worker 取走；
3. 每个项目处理完并调用 `task_done()`；
4. 未完成计数归零后，`join()` 返回；
5. Queue 排空后，等待或再次调用 `get()` 的 worker 收到 `QueueShutDown` 并退出；
6. `TaskGroup` 等协调者和所有 worker 结束后，父作用域才读取最终结果。

空输入也走同一个协议：关闭空 Queue 后，等待 `get()` 的 worker 会收到 `QueueShutDown`，不需要额外的空列表特判。

注意，先 `shutdown()` 再 `join()` 并不矛盾。默认关闭只停止继续入队，不会丢弃已经排队的项目。

异常或用户取消：放弃剩余工作并解除等待

若 worker 崩溃、生产者失败，或用户取消父请求，继续等待全部业务完成通常已经没有意义。协调者的 `finally` 可以执行：

```
queue.shutdown(immediate=True)
```

紧急关闭会清空尚未领取的项目，并解除阻塞在 `put()` 或 `get()` 的调用。被 worker 领取的项目完成记账后，未完成计数才会归零并解除仍在等待的 `join()`。它用于让失败路径有限时收束，不是成功信号。此时 `join()` 的通常不变量被削弱：未执行的排队项目也可能因被清空而不再阻止它返回。

已经被 worker 领取的项目不在 Queue 中。它们仍要在 worker 的 `finally` 中释放本地资源并调用 `task_done()`。紧急关闭也不能撤销已经发往远端的请求、数据库提交或其他副作用。

TaskGroup 才是所有者

一条完整管道应有清楚的所有权：

```
run_pipeline: 拥有本次批处理
│
├── Queue: 保存等待领取的普通数据，并由本函数关闭
├── TaskGroup: 拥有全部长期 Task
│   ├── producer/coordinator: 生产、正常 shutdown、等待 join
│   ├── worker-0: 一次处理一个已领取项目
```

DME × | readme × | README × | README × | README × | README × | README × | README × | README × | README × | README ×
2502x6dg8fpjj0sff40000gp/T/crossnote-f-4ac7ee220d97d8ab8b8febd0c61.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

TaskGroup 才是所有者

一条完整管道应有清楚的所有权：

```
run_pipeline：拥有本次批处理
│
├── Queue：保存等待领取的普通数据，并由本函数关闭
├── TaskGroup：拥有全部长期 Task
│   ├── producer/coordinator：生产、正常 shutdown、等待 join
│   ├── worker-0：一次处理一个已领取项目
│   ├── worker-1：一次处理一个已领取项目
│   └── worker-2：一次处理一个已领取项目
└── 父作用域：仅在 TaskGroup 成功退出后合并最终结果
```

异常策略也应与所有权对齐：

情况	子任务怎样处理	整批结果
一项可预期业务失败	转为带错误字段的结果，照常 `task_done()`	其他项目继续，可得到部分失败结果
worker 程序错误	不吞异常，让它离开 worker	TaskGroup 取消同伴，整批失败
生产者无法继续读取输入	不伪造“输入结束”	TaskGroup 取消 worker，整批失败
用户取消父 Task	`finally` 清理并继续传播取消	调用方仍收到 `CancelledError`

只捕获已经定义好恢复语义的业务异常。若 worker 用 `except Exception` 把解析器缺陷、断言失败等全部包装成普通结果，TaskGroup 就看不到失败，也无法启动兄弟取消和统一清理。

Python 3.12 及更旧版本：哨兵协议

旧版本没有 `Queue.shutdown()`，常见替代方案是在所有真实项目之后，为每个 worker 放一个不会与业务数据混淆的哨兵：

```
STOP = object()

for _ in range(worker_count):
    await queue.put(STOP)
```

三个 worker 需要三个哨兵；哨兵经过 `get()` 后也要配对 `task_done()`。一个哨兵不会自动广播。更重要的是，生产者失败时可能根本来不及投递哨兵，甚至会卡在向满队列投递哨兵的途中，所以

Python 3.12 及更旧版本：哨兵协议

旧版本没有 `Queue.shutdown()`，常见替代方案是在所有真实项目之后，为每个 worker 放一个不会与业务数据混淆的哨兵：

```
STOP = object()

for _ in range(worker_count):
    await queue.put(STOP)
```

三个 worker 需要三个哨兵；哨兵经过 `get()` 后也要配对 `task_done()`。一个哨兵不会自动广播。更重要的是，生产者失败时可能根本来不及投递哨兵，甚至会卡在向满队列投递哨兵的途中，所以异常路径仍要靠 TaskGroup 取消并等待 worker。

23.8 单线程事件循环仍然会发生逻辑竞争

在同一个事件循环线程内，一个 Task 只会在可暂停点把执行权交还给事件循环。因此不跨 `await` 的一小段同步代码通常不会被另一个 asyncio Task 插入。

但只要一个不变量被拆在 `await` 两侧，就可能发生丢失更新：

```
snapshot = counter.value
await something()
counter.value = snapshot + 1
```

假设两个 Task 都先读到 0，然后都在 `await` 处暂停。它们恢复后会分别把 1 写回同一个字段，两次加一最终只得到 1。动态调度是否经常触发并不重要；只要存在这种交错，代码就不正确。

Event 只能协调阶段

`asyncio.Event` 保存一个布尔状态：

- 初始未设置时，`wait()` 会暂停；
- `set()` 后，所有当前等待者都可继续，以后的 `wait()` 也会立即返回；
- `clear()` 后，未来的 `wait()` 再次需要等待。

它适合表达“所有 worker 已启动”“两个 Task 都完成读取”“现在允许进入下一阶段”。它不会限制只有一个 Task 修改状态，因此不是互斥锁。本章示例正是用 Event 让两个 Task 确定都读到旧值，

ADME × | readme × | README × | README × | README × | README × | README × | README × | README × | README × | README ×
2502x6dg8fpjj0sff40000gp/T/crossnote-f-4ac7ee220d97d8ab8b8febd0c61.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

Event 只能协调阶段

`asyncio.Event` 保存一个布尔状态：

- 初始未设置时，`wait()` 会暂停；
- `set()` 后，所有当前等待者都可继续，以后的 `wait()` 也会立即返回；
- `clear()` 后，未来的 `wait()` 再次需要等待。

它适合表达“所有 worker 已启动”“两个 Task 都完成读取”“现在允许进入下一阶段”。它不会限制只有一个 Task 修改状态，因此不是互斥锁。本章示例正是用 Event 让两个 Task 确定都读到旧值，从而稳定复现竞争，而不是用随机 `sleep()` 碰运气。

必须共享一个不变量时使用 Lock

```
async with counter_lock:
    snapshot = counter.value
    counter.value = snapshot + 1
```

`asyncio.Lock` 同一时刻只允许一个 Task 进入临界区，离开 `async with` 时释放。锁应包住必须一起观察和修改的不变量，而不是习惯性包住整个函数。

尤其不要无条件持锁等待慢网络：

```
async with state_lock:
    response = await slow_network_call()  # 所有状态使用者一起排队
    state[key] = response
```

若网络调用与共享不变量无关，先在锁外完成 I/O，再短暂持锁提交结果：

```
response = await slow_network_call()

async with state_lock:
    state[key] = response
```

但这不是机械规则。若业务要求“按版本读取、发请求、仅在版本未变化时提交”，完整不变量跨越了网络等待，应使用版本检查、不可变快照或单一状态所有者，而不是直接拆锁。

优先消除共享写入

DME × | readme × | README × | README × | README × | README × | README × | READMI × | READMI × | READMI × | READM
2502x6dg8fpjj0sff40000gp/T/crossnote-f-4ac7ee220d97d8ab8b8febd0c61.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

优先消除共享写入

本章完整示例让每个 worker 只写自己的结果列表。所有 worker 退出后，父任务再接管并合并结果。另一种做法是让子任务返回局部值，由父任务在 TaskGroup 结束后求和。两种方式都不需要多个 Task 共同读改写一个聚合器。

选择顺序可以记成：

1. 能让一个 Task 单独拥有状态，就先改变所有权；
2. 必须共享同一个不变量时，才用 Lock 保护最小但完整的临界区；
3. Event 只传递阶段事实，不提供互斥；
4. 若持锁期间还要 `await`，画出“谁持锁、正在等待谁”，检查是否可能循环等待。

23.9 常见迁移错误与排查顺序

错误判断或写法	典型表现	正确方向
`Semaphore(3)` 等于每秒 3 次	快请求一秒内远超 3 次	单独设计带时间窗口的速率限制
给海量项目逐个建 Task 再套 Semaphore	在途很少，但 Task 和内存仍很多	有界 Queue + 固定 worker
`Queue()` 不设正数 `maxsize`	快生产者持续堆积内存	明确容量，让 `await put()` 产生背压
成功路径才调用 `task_done()`	异常或取消后 `join()` 永久等待	成功 `get()` 后立即建立 `try/finally`
`get()` 失败也调用 `task_done()`	抛 `ValueError` 或账本失真	只为实际领取的项目记完成
`join()` 返回就认为 worker 已退出	TaskGroup 一直等在 worker 的下次 `get()`	再设计 shutdown 或足量哨兵
三个 worker 只投一个哨兵	一个退出，两个永久等待	每 worker 一个哨兵，或使用 Queue shutdown
worker 捕获所有 `Exception`	程序缺陷被伪装成业务失败	只恢复明确的业务异常
生产者失败后仍等待正常 `join()`	剩余项目无人处理，管道死等	让异常离开 Task，由 TaskGroup 联动取消和紧急关闭
把紧急 shutdown 当成处理完成	部分项目从未执行	只把它用于放弃和清理路径
认为单线程就没有竞争	跨 `await` 的状态更新丢失	改变所有权，或用 Lock 保护不变量

ОМЕ × | readme × | README × | README × | README × | README × | README × | READMI × | READMI × | READMI × | READMI ×
2502x6dg8fpjj0sff40000gp/T/crossnote-f-4ac7ee220d97d8ab8b8febd0c61.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

认为单线程就没有竞争	跨 `await` 的状态更新丢失	改变所有权，或用 Lock 保护不变量
持锁等待无关慢 I/O	其他 Task 长时间排队	缩短临界区并重新审视状态协议

Queue 管道挂住时，按下面顺序检查：

1. 每次成功 `put()` 的项目最终是否被某个 worker `get()`，或在紧急关闭时被明确丢弃？
2. 每次成功 `get()` 是否在所有返回、异常和取消路径恰好调用一次 `task_done()`？
3. `join()` 返回后，worker 是否还能收到关闭信号？
4. 生产者失败时，是否还有 worker 在等永远不会到来的数据或哨兵？
5. 所有长期 Task 是否属于同一个 TaskGroup？
6. 是否有过宽的异常捕获阻止 TaskGroup 感知真正失败？

23.10 递进练习

练习 1：证明 Semaphore 只限制在途数量

并发查询 6 个本地替身，许可数为 2。使用 Event 让已经进入的调用暂停，再断言只有 2 个调用进入受限区，但已经创建了 6 个 Task。取消一个持有许可的 Task 后，再证明等待者可以取得被归还的许可。

验收点：

- 使用 `async with semaphore`，不访问私有计数器；
- 最大在途为 2，记录 Task 数仍为 6；
- 不用墙钟延迟猜测调度阶段；
- 能解释为什么该实验没有证明“每秒最多 2 次”。

练习 2：实现一个有界 worker 池

用 2 个固定 worker 处理 `range(10)`，Queue 容量为 3，每项结果为平方。生产完成后使用正常 `shutdown()` 和 `join()`，最后由父任务按输入顺序整理结果。

验收点：

- 长期基础设施 Task 数不随输入项数增长；
- 任意时刻 `qsize() <= 3`；
- 每次成功 `get()` 都在 `finally` 中配对一次 `task_done()`；
- `join()` 返回后，两个 worker 也都已经退出；

验收点：

- 长期基础设施 Task 数不随输入项数增长；
- 任意时刻 `qsize() <= 3`；
- 每次成功 `get()` 都在 `finally` 中配对一次 `task_done()`；
- `join()` 返回后，两个 worker 也都已经退出；
- 最终结果是 `[0, 1, 4, ..., 81]`。

练习 3：覆盖三种失败路径

分别注入生产者读取失败、worker 程序错误和用户取消。可恢复业务失败应转为结果；另外三种情况都必须让 TaskGroup 收束，并且不能返回看似完整的成功列表。

验收点：

- 程序错误不会被包装成普通业务结果；
- 非正常路径使用紧急关闭，但不声称剩余工作已完成；
- 所有 worker 的 `finally` 都已运行；
- 用户取消最终仍以 `CancelledError` 传播；
- 事件循环中没有遗留后台 Task。

练习 4：比较 Lock 与状态所有权

让两个 Task 各执行一次“读计数— `await` —写计数”。先用 Event 稳定构造丢失更新，再分别用 Lock 和“子任务返回局部结果、父任务合并”修复。

验收点：

- 能指出哪个 `await` 拆开了读改写；
- Lock 版得到正确总数；
- 所有权版不共享可变计数器；
- 能解释 Event 为什么不能替代 Lock。

23.11 本章小结

把本章压缩成一条设计顺序：

1. 先确定受限资源和单位：在途数、速率、排队数还是连接数；
2. 小批量用 TaskGroup 管生命周期、Semaphore 管受限区；

README × | readme × | README × | README × | README × | README × | README × | README × | README × | README ×
vf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-4ac7ee220d97d8ab8b8febd0c61.html
相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

23.11 本章小结

把本章压缩成一条设计顺序：

1. 先确定受限资源和单位：在途数、速率、排队数还是连接数；
2. 小批量用 TaskGroup 管生命周期、Semaphore 管受限区；
3. 大量或流式输入用有界 Queue 承载等待工作、固定 worker 控制 Task 数；
4. 把 Queue 的容量、完成记账和 worker 关闭当成三套协议分别设计；
5. 正常路径排空，失败和取消路径放弃剩余工作并确定性清理；
6. 跨 `await` 的共享读改写仍会竞争，优先改变所有权，必要时再加 Lock。

只要能回答下面五个问题，就已经建立了本章的核心心智模型：

1. 为什么 `Semaphore(3)` 不能保证每秒最多 3 次请求？
2. 为什么 10,000 个等待 Semaphore 的 Task 仍然占用 10,000 份 Task 状态？
3. `Queue(maxsize=4)` 限制了哪些数据，又没有限制哪些内存？
4. `task_done()`、`join()` 和 `shutdown()` 分别负责什么？
5. worker、生产者或父请求失败时，谁传播异常、谁关闭 Queue、谁等待所有 Task？

23.12 运行本章示例

本章只有两个文件：

文件	作用
`README.md`	建立并发容量、背压、生命周期和共享状态的完整模型
`example.py`	用可控 Event 和运行时断言验证正常、异常、取消与竞争路径

根 `pyproject.toml` 要求 Python `>=3.14`。本章只使用 `asyncio` 和 `dataclasses`，不需要安装额外依赖。请在仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/23_asyncio_concurrency_control/example.py
```

`-B` 只阻止生成 `.pyc`，不改变程序行为。`java_to_python/run_all.py` 只包含前 10 章，因此不能用它运行本章。

完整示例会依次验证：

README × | readme × | README × | README × | README × | README × | README × | READMI × | READMI × | READMI × | READM
rf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-4ac7ee220d97d8ab8b8febd0c61.html
关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

本章只有两个文件：

文件	作用
`README.md`	建立并发容量、背压、生命周期和共享状态的完整模型
`example.py`	用可控 Event 和运行时断言验证正常、异常、取消与竞争路径

根 `pyproject.toml` 要求 Python `>=3.14`。本章只使用 `asyncio` 和 `dataclasses`，不需要安装额外依赖。请在仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/23_asyncio_concurrency_control/example.py
```

`-B` 只阻止生成 `.pyc`，不改变程序行为。`java_to_python/run_all.py` 只包含前 10 章，因此不能用它运行本章。

完整示例会依次验证：

- 四种容量限制的单位不同；
- 等待许可和持有许可时取消都不会泄漏 Semaphore 许可；
- 9 项小批量创建 9 个 Task，但最大在途为 3；
- 24 项输入经过容量为 4 的 Queue 和 3 个 worker，生产者确实遇到背压；
- 空输入、worker 崩溃、生产者崩溃和用户取消都能让 worker 退出；
- 漏掉 `task_done()` 会卡住 `join()`，而 `join()` 返回也不会自动关闭 worker；
- 两个 Task 跨 `await` 修改共享计数时会丢失更新，Lock 与局部所有权都能修复。

运行静态检查和格式检查：

```
.venv/bin/ruff check java_to_python/chapters/23_asyncio_concurrency_control
.venv/bin/ruff format --check java_to_python/chapters/23_asyncio_concurrency_control
.venv/bin/pyright java_to_python/chapters/23_asyncio_concurrency_control/example.py
```

进一步核对 API 细节可查阅 Python 3.14 官方文档：

- `Semaphore`、`Lock` 与 `Event`；
- `Queue`、`task_done()`、`join()` 与 `shutdown()`；
- `TaskGroup`。

```python
"""第 23 章：并发上限、Queue、背压与共享状态。

所有“查询”都是纯内存替身。Event 控制关键阶段，sleep(0) 只提供一次事件循环调度点；
示例不访问真实网络，也不依赖某段墙钟时间内恰好完成多少工作。
"""

# 阅读顺序：先看 Semaphore 如何约束“已经进入查询”的 Task，再看 Queue 如何把“尚未领取”
# 的工作限制在固定容量内，最后用一个确定性实验观察跨 await 的共享状态竞争。
# 数据主线都是 Record -> 替身 query -> QueryResult；assert 负责把并发与清理规则变成可验证结果。

# asyncio 提供协程、Task、TaskGroup 以及只供事件循环 Task 使用的同步原语。
# dataclass 用来生成保存输入、结果和指标的轻量对象；field 在每个实例构造时创建独立的可变字段。
import asyncio
from dataclasses import dataclass, field

# 这三个常量分别约束不同位置：Semaphore 的在途调用、Queue 中等待领取的记录、长期 worker 数。
# 数值相同不代表含义相同；修改其中一个时，不应默认另外两个也要随之变化。
MAX_IN_FLIGHT = 3
QUEUE_CAPACITY = 4
WORKER_COUNT = 3


# 三种异常把“可继续的单项业务失败”与“必须让整个 TaskGroup 失败”的基础设施错误分开。
# worker 只会捕获第一种；后两种保持向外传播，由 TaskGroup 联动取消兄弟 Task。
class RecordNotFound(Exception):
    """一次可预期的业务失败：本批次可以继续处理其他记录。"""


class WorkerCrashed(RuntimeError):
    """worker 内部出现未知程序错误：整个任务组必须失败并清理。"""


class ProducerCrashed(RuntimeError):
    """生产者无法继续枚举输入：整个任务组必须失败并清理。"""


# dataclass 类似减少了 Java record/POJO 的样板代码，会按字段生成初始化和比较等方法。
```

```
36
37  # dataclass 类似减少了 Java record/POJO 的样板代码，会按字段生成初始化和比较等方法。
38  # frozen=True 禁止普通字段重新赋值，slots=True 不再为每个实例创建任意扩展属性的 __dict__；
39  # 它们是 Python 运行时的数据模型选项，而下面的 int/str 类型提示主要仍由类型检查器检查。
40  @dataclass(frozen=True, slots=True)
41  class Record:
42      # record_id 用于稳定匹配结果和故障开关；query 模拟已经准备好的查询文本。
43      record_id: int
44      query: str
45
46
47  @dataclass(frozen=True, slots=True)
48  class QueryResult:
49      # str | None 表示“字符串或 None”。成功时 value 有值而 error 为 None，失败时正好相反。
50      record_id: int
51      value: str | None
52      error: str | None
53
54      # @property 让调用者写 result.ok，而不是 result.ok()；每次访问都会执行下面的判断。
55      @property
56      def ok(self) -> bool:
57          # None 是单例，用 is 判断身份；不要照搬 Java 写成 error == null。
58          return self.error is None
59
60
61  @dataclass(slots=True)
62  class ConcurrencyProbe:
63      """记录替身查询的在途数量，并用 Event 构造可重复的并发阶段。"""
64
65      # expected_peak 是本次实验必须会合的调用数；达到前所有已进入的 query 都会等待。
66      expected_peak: int
67      # 传入额外 Event 时，即使达到峰值也继续暂停，供“父任务取消”实验精确控制时机。
68      hold_after_peak: asyncio.Event | None = None
69      # active 是当前尚未离开 query 的数量，max_active 保存整个实验见过的峰值。
70      active: int = 0
71      max_active: int = 0
72      # started_count/cleaned_count 和 ID 列表用于核对开始、清理及实际处理过的记录。
```

```
62  class ConcurrencyProbe:
72      # started_count/cleaned_count 和 ID 列表用于核对开始、清理及实际处理过的记录。
73      started_count: int = 0
74      cleaned_count: int = 0
75      # default_factory=list 会为每个 ConcurrencyProbe 实例调用一次 list()；避免多个实例共享同一列表。
76      started_ids: list[int] = field(default_factory=list)
77      # asyncio.Event 同理必须逐实例创建。它是“一旦置位即可放行所有等待者”的阶段信号，不是锁。
78      peak_reached: asyncio.Event = field(default_factory=asyncio.Event)
79
80      # async def 调用后先得到 coroutine 对象；由 await 或 Task 驱动时，函数体才真正向前执行。
81      # record 是本次输入；星号后的两个故障 ID 必须以关键字传入，None 表示不启用该故障。
82      async def query(
83          self,
84          record: Record,
85          *,
86          business_failure_id: int | None = None,
87          crash_id: int | None = None,
88      ) -> str:
89          # 这些计数更新之间没有 await；在同一个事件循环线程里不会被别的 Task 插入。
90          # 这只保证这一小段连续执行，不代表整个 async 函数或任意共享操作天然是原子的。
91          self.active += 1
92          self.started_count += 1
93          self.started_ids.append(record.record_id)
94          self.max_active = max(self.max_active, self.active)
95          try:
96              # 超限立即把测试判为失败；等于目标值时 set()，唤醒所有等待 peak_reached 的 Task。
97              if self.active > self.expected_peak:
98                  raise AssertionError(f"在途查询超过上限：{self.active}")
99              if self.active == self.expected_peak:
100                 self.peak_reached.set()
101
102             # 前 expected_peak 个调用会在这里会合，保证实验确实观察到峰值，而非猜调度顺序。
103             # Event 一旦 set，当前和以后调用 wait() 的 Task 都可继续；它不会一次只放行一个。
104             await self.peak_reached.wait()
105             # is not None 明确区分“没有第二道闸门”和“传入了一个尚未 set 的 Event”。
106             if self.hold_after_peak is not None:
107                 await self.hold_after_peak.wait()
```

```
 62  class ConcurrencyProbe:
 82      async def query(
106              if self.hold_after_peak is not None:
107                  await self.hold_after_peak.wait()
108
109              # 代表一次会把控制权交回事件循环的 I/O；0 表示不用墙钟延迟做正确性判断。
110              await asyncio.sleep(0)
111              # 两个比较按 record_id 注入确定性结果：业务失败会被上层转成数据，崩溃则向外传播。
112              if record.record_id == business_failure_id:
113                  raise RecordNotFound(f"记录 {record.record_id} 不存在")
114              if record.record_id == crash_id:
115                  raise WorkerCrashed(f"处理记录 {record.record_id} 时解析器崩溃")
116              return f"记录 {record.record_id} 的结果"
117          finally:
118              # 真实客户端应在这里归还连接或关闭响应体；取消和异常也必须经过这里。
119              # finally 覆盖正常 return、两类异常以及 await 处收到的 CancelledError。
120              self.active -= 1
121              self.cleaned_count += 1
122
123
124  @dataclass(slots=True)
125  class QueueMetrics:
126      # 这些字段分别观察容量、背压以及 worker/Queue 的生命周期；默认值使每次运行从 0 开始。
127      max_queued: int = 0
128      backpressure_waits: int = 0
129      workers_started: int = 0
130      workers_exited: int = 0
131      graceful_shutdowns: int = 0
132      immediate_shutdowns: int = 0
133
134
135  @dataclass(slots=True)
136  class Counter:
137      # 可变计数器专门用于最后的竞争实验；两个 Task 会持有同一个 Counter 实例。
138      value: int = 0
139
```

```python
@dataclass(slots=True)
class Counter:
    # 可变计数器专门用于最后的竞争实验；两个 Task 会持有同一个 Counter 实例。
    value: int = 0


# 普通 def 会立即执行，不会产生 coroutine；-> None 表示函数只负责输出，没有有意义的返回值。
def print_section(title: str) -> None:
    # f-string 在运行时计算花括号中的 title；开头的 \n 先输出一个空行分隔实验。
    print(f"\n=== {title} ===")


def make_records(count: int) -> list[Record]:
    # 列表推导式遍历 range(0, count)，每轮用关键字参数创建一个 Record，最终返回完整 list。
    # 它更接近 Java Stream 的 map(...).toList()，但这里是立即求值，不是惰性输入流。
    return [Record(record_id=index, query=f"查询-{index}") for index in range(count)]


def explain_independent_limits() -> None:
    # 本段只建立术语边界：在途、单位时间请求、排队和连接占用是四个可独立配置的维度。
    print_section("1. 四种限制解决四个不同问题")
    print("  在途并发上限：同一时刻最多多少次调用尚未结束")
    print("  请求速率上限：某个时间窗口最多允许多少次开始请求")
    print("  Queue 容量：最多允许多少项尚未被 worker 取走")
    print("  HTTP 连接池：客户端最多建立或借出多少连接，并负责连接复用")
    print("  结论：Semaphore 只管同时占用许可的数量，不会等待下一个时间窗口。")


async def hold_one_permit(
    limit: asyncio.Semaphore,
    entered: asyncio.Event,
    cleaned: asyncio.Event,
) -> None:
    # async with 会先 await limit.acquire()，进入后独占这唯一许可，退出时自动 release()。
    async with limit:
        # set() 本身不暂停：它通知父任务“许可已经拿到”，然后本 Task 继续向下运行。
```

```python

async def hold_one_permit(
    limit: asyncio.Semaphore,
    entered: asyncio.Event,
    cleaned: asyncio.Event,
) -> None:
    # async with 会先 await limit.acquire()，进入后独占这唯一许可，退出时自动 release()。
    async with limit:
        # set() 本身不暂停：它通知父任务“许可已经拿到”，然后本 Task 继续向下运行。
        entered.set()
        try:
            # 这个临时 Event 永远不会被 set，因此 wait() 会一直挂起；实验只通过 cancel() 结束它。
            await asyncio.Event().wait()
        finally:
            # 取消从 wait() 抛入后先执行这里，再离开 async with 并归还 Semaphore 许可。
            cleaned.set()


async def wait_for_one_permit(
    limit: asyncio.Semaphore,
    attempting: asyncio.Event,
    entered: asyncio.Event,
) -> None:
    # 先通知父任务“马上要申请许可”，便于父任务确定取消发生在等待 acquire 的阶段。
    attempting.set()
    async with limit:
        # 只有 acquire 真正成功才会执行；等待期间取消时，这个 Event 应始终未设置。
        entered.set()


async def demonstrate_semaphore_cancellation() -> None:
    print_section("2. Semaphore: 等待许可和持有许可时被取消都不泄漏")
    # 许可数为 1，先启动的 holder 会让后启动的 waiter 必然停在 acquire，而不是碰运气竞争。
    limit = asyncio.Semaphore(1)
    # 每个 Event 都表示一个单向阶段事实；名称说明由谁设置、父任务等待观察什么。
    holder_entered = asyncio.Event()
    holder_cleaned = asyncio.Event()
```

```python

async def demonstrate_semaphore_cancellation() -> None:
    print_section("2. Semaphore: 等待许可和持有许可时被取消都不泄漏")
    # 许可数为 1，先启动的 holder 会让后启动的 waiter 必然停在 acquire，而不是碰运气竞争。
    limit = asyncio.Semaphore(1)
    # 每个 Event 都表示一个单向阶段事实；名称说明由谁设置、父任务等待观察什么。
    holder_entered = asyncio.Event()
    holder_cleaned = asyncio.Event()
    waiter_attempting = asyncio.Event()
    waiter_entered = asyncio.Event()

    # TaskGroup 是这些 Task 的结构化所有者：离开作用域前会等待它们全部结束。
    async with asyncio.TaskGroup() as group:
        # create_task 把 coroutine 纳入事件循环调度，并返回可取消、可等待的 Task 句柄。
        holder: Task[None] = group.create_task(
            hold_one_permit(limit, holder_entered, holder_cleaned),
            name="permit-holder",
        )
        # 等 holder 明确发出“已进入”信号，下面才创建 waiter，避免依赖创建先后猜调度结果。
        await holder_entered.wait()

        waiter: Task[None] = group.create_task(
            wait_for_one_permit(limit, waiter_attempting, waiter_entered),
            name="permit-waiter",
        )
        # attempting 在 acquire 前设置；当前许可仍由 holder 占用，所以 waiter 随后必定等待。
        await waiter_attempting.wait()
        # cancel() 只是发出取消请求；继续 await 才能观察 CancelledError 并确认清理已经结束。
        waiter.cancel()
        try:
            await waiter
        except asyncio.CancelledError:
            # 本例有意消费已确认的子 Task 取消；生产代码不应无意吞掉当前父 Task 的取消。
            pass

        # waiter 没进入上下文体，就没有获取许可，也不存在需要它自行释放的许可。
        assert not waiter_entered.is_set()
```

```
225
226         # waiter 没进入上下文，就没有获取许可，也不存在需要它自行释放的许可。
227         assert not waiter_entered.is_set()
228         # 再取消已经持有许可的 holder，验证 async with 的退出路径会自动归还许可。
229         holder.cancel()
230         try:
231             await holder
232         except asyncio.CancelledError:
233             pass
234
235     # TaskGroup 已等待两个 Task 完整退出；此时检查 finally 的副作用不会再与子 Task 并发变化。
236     assert holder_cleaned.is_set()
237     # holder 的 async with 已在取消展开时释放许可；这里应能立刻重新进入。
238     async with limit:
239         reacquired = True
240
241     # 三个输出依次证明：等待者未进入、持有者已清理、同一个 Semaphore 仍可使用。
242     print("  等待中的 Task 未误入临界区:", not waiter_entered.is_set())
243     print("  持有者取消后执行 finally:", holder_cleaned.is_set())
244     print("  取消清理后可以重新获取许可:", reacquired)
245
246
247 async def query_with_limit(
248     record: Record,
249     limit: asyncio.Semaphore,
250     probe: ConcurrencyProbe,
251     *,
252     business_failure_id: int | None,
253 ) -> QueryResult:
254     # 许可尽量贴近真正受限的 I/O；参数准备和结果格式化不必占用稀缺许可。
255     # 每个记录 Task 都会调用本函数，但同时最多 MAX_IN_FLIGHT 个 Task 能进入 probe.query。
256     try:
257         async with limit:
258             value: str = await probe.query(record, business_failure_id=business_failure_id)
259     except RecordNotFound as error:
260         # 只把有修复约定的业务异常转成 QueryResult；未知异常和取消仍交给 TaskGroup 处理。
```

```python
async def query_with_limit(
    except RecordNotFound as error:
        # 只把有恢复约定的业务异常转成 QueryResult；未知异常和取消仍交给 TaskGroup 处理。
        # str(error) 取异常消息，value=None 与 error 字符串共同表示这一项失败。
        return QueryResult(record.record_id, value=None, error=str(error))
    # try 正常完成时 value 已绑定为查询字符串；构造成功结果时 error 明确为 None。
    return QueryResult(record.record_id, value=value, error=None)


async def run_small_semaphore_batch(
    records: list[Record],
) -> tuple[list[QueryResult], ConcurrencyProbe, int]:
    # 返回的是三元组：(按输入 Task 顺序的结果列表，探针对象，实际创建的记录 Task 数)。
    limit = asyncio.Semaphore(MAX_IN_FLIGHT)
    # 非空小批次不足 3 项时，等待实际可达到的峰值，不能让替身自己制造死锁。
    expected_peak: int = max(1, min(MAX_IN_FLIGHT, len(records)))
    probe = ConcurrencyProbe(expected_peak=expected_peak)

    async with asyncio.TaskGroup() as group:
        # TaskGroup 管生命周期；Semaphore 管进入 query 的并发数。
        # 因此 N 条记录仍会在这里创建 N 个 Task。
        # 推导式会立刻为 records 中每条记录调用 create_task；Semaphore 不会推迟 Task 的创建。
        # tasks 保存句柄，是为了等 TaskGroup 成功退出后按创建顺序读取每个 result()。
        tasks: list[Task[QueryResult]] = [
            group.create_task(
                query_with_limit(
                    record,
                    limit,
                    probe,
                    business_failure_id=2,
                ),
                name=f"small-batch-{record.record_id}",
            )
            for record in records
        ]
```

```
293
294     # 走到这里表示 TaskGroup 已成功等待所有任务；result() 此时不会阻塞，只取已完成的返回值。
295     return [task.result() for task in tasks], probe, len(tasks)
296
297
298 async def demonstrate_small_semaphore_batch() -> None:
299     print_section("3. 小批量 + Semaphore：限制在途，但 Task 已全部创建")
300     records: list[Record] = make_records(9)
301     results: list[QueryResult], probe: ConcurrencyProbe, created_tasks: int = await run_small_semaphore_batch(record
302
303     # 这组断言同时验证“Task 数没有受限”“查询并发受限”“finally 清零在途计数”。
304     assert created_tasks == len(records)
305     assert probe.max_active == MAX_IN_FLIGHT
306     assert probe.active == 0
307     # bool 在求和中按 1/0 计算，因此这里是在统计 ok 为 True 的结果数量。
308     assert sum(result.ok for result in results) == 8
309     print(f"  输入={len(records)}，创建的记录 Task={created_tasks}")
310     print(f"  在途上限={MAX_IN_FLIGHT}，观察到的最大在途={probe.max_active}")
311     print("  业务结果：成功 8，失败 1；批次没有因可预期失败而中止")
312
313
314 async def queue_worker(
315     worker_id: int,
316     queue: asyncio.Queue[Record],
317     own_results: list[QueryResult],
318     probe: ConcurrencyProbe,
319     metrics: QueueMetrics,
320     *,
321     business_failure_id: int | None,
322     crash_id: int | None,
323 ) -> None:
324     """一个固定 worker；own_results 只由这个 worker 写，退出后交还父作用域。"""
325     # worker_id 只用于日志；queue 传递待处理 Record；own_results 是当前 worker 独占的结果列表。
326     # probe/metrics 记录可观测状态；两个可空 ID 决定替身在哪条记录注入业务失败或程序崩溃。
327     metrics.workers_started += 1
328     try:
```

```
async def queue_worker(
) -> None:
    """一个固定 worker；own_results 只由这个 worker 写，退出后交还父作用域。"""
    # worker_id 只用于日志；queue 传递待处理 Record；own_results 是当前 worker 独占的结果列表。
    # probe/metrics 记录可观测状态；两个可空 ID 决定替身在哪条记录注入业务失败或程序崩溃。
    metrics.workers_started += 1
    try:
        # worker 长期存在，不为每条记录新建 Task；正常和紧急 shutdown 都通过异常结束此循环。
        while True:
            try:
                # Queue 为空时 get() 会暂停；成功返回后 record 已从"排队"转为此 worker 的在途工作。
                record: Record = await queue.get()
            except asyncio.QueueShutDown:
                # 正常 shutdown 后，队列排空；或紧急 shutdown 后，worker 被唤醒。
                # get() 没有交付项目，所以这个分支绝不能调用 task_done()。
                return

            # 外层 try/finally 从成功 get 的这一刻开始，确保无论处理怎样结束都完成一次 Queue 记账。
            try:
                try:
                    value: str = await probe.query(
                        record,
                        business_failure_id=business_failure_id,
                        crash_id=crash_id,
                    )
                except RecordNotFound as error:
                    # 只转换已定义恢复语义的业务错误；WorkerCrashed 和取消继续传播。
                    # append 之前没有 await，且此列表没有其他写入者，因此不需要 Lock。
                    own_results.append(QueryResult(record.record_id, value=None, error=str(error)))
                else:
                    # try 的 else 只在没有异常时执行，可保证 value 已由 await 的返回值成功绑定。
                    own_results.append(QueryResult(record.record_id, value=value, error=None))
            finally:
                # 每次成功 get 都恰好对应一次 task_done，包括业务失败、程序异常和取消。
                # task_done() 不会删除元素；它递减 put() 建立的"未完成工作"计数，使 join() 可返回。
                queue.task_done()
    finally:
```

```
async def queue_worker(
                queue.task_done()
    finally:
        # 这一层覆盖 shutdown、WorkerCrashed 和 TaskGroup 取消，用于证明每个长期 worker 都已回收。
        metrics.workers_exited += 1
        print(f"  worker-{worker_id} 已退出")


async def produce_then_join(
    queue: asyncio.Queue[Record],
    records: list[Record],
    metrics: QueueMetrics,
    *,
    producer_fail_after: int | None,
) -> None:
    """生产者兼正常关闭协调者；异常路径用紧急关闭解除 Queue 等待。"""
    # False 表示尚未完成"正常关闭 + 所有已入队项目记账归零"的整套协议。
    # 不能只看是否调用过 shutdown，因为等待 join 时仍可能发生 worker 崩溃或用户取消。
    joined_normally = False
    try:
        # enumerate 同时产生从 0 开始的索引和当前 Record，类似带计数器的 Java for-each。
        for index, record in enumerate(records):
            # 这个开关模拟流式输入源在读取下一项前损坏；None 表示不注入故障。
            if producer_fail_after is not None and index == producer_fail_after:
                raise ProducerCrashed(f"读取到第 {index} 项时输入源损坏")

            if queue.full():
                # full() 与紧接着的 put() 之间没有 await；put 会在满队列上形成真实背压。
                # 这里的计数只证明本例观察过满队列；真实正确性仍来自 await put() 自身的等待语义。
                metrics.backpressure_waits += 1
            # Queue 满时生产者在此暂停，直到某个 worker get() 腾出位置；记录不会被静默丢弃。
            await queue.put(record)
            # qsize() 是此刻快照。当前到 max() 之间没有 await，适合更新教学指标；
            # 若先 await 再用这个值做并发协议，其他 Task 可能已改变队列，结论就不再成立。
            metrics.max_queued = max(metrics.max_queued, queue.qsize())

```

```python
async def produce_then_join(

        # 不再接收新工作，但允许 worker 取完已入队项目；空批次也会直接唤醒 worker。
        queue.shutdown()
        metrics.graceful_shutdowns += 1
        # join() 等未完成计数归零，不负责等待 worker 退出；worker 由上面的 shutdown 得到退出信号。
        await queue.join()
        # 只有 join 确实返回才标记正常完成，使 finally 能区分成功与任意中断路径。
        joined_normally = True
    finally:
        if not joined_normally:
            # worker 崩溃、生产者崩溃或用户取消时，不等待“处理完所有业务”。
            # immediate=True 会丢弃尚未领取的项目，所以它是放弃路径，不是成功证明。
            # 紧急关闭还能唤醒正等待 put()/get()/join() 的相关 Task，让 TaskGroup 有限时收尾。
            queue.shutdown(immediate=True)
            metrics.immediate_shutdowns += 1


async def run_worker_queue(
    records: list[Record],
    *,
    business_failure_id: int | None = None,
    crash_id: int | None = None,
    producer_fail_after: int | None = None,
    probe: ConcurrencyProbe | None = None,
    metrics: QueueMetrics | None = None,
) -> tuple[list[QueryResult], ConcurrencyProbe, QueueMetrics, int]:
    """拥有 Queue、TaskGroup 和结果聚合；成功退出后才把结果交给调用者。"""
    # Queue[Record] 是静态类型信息；maxsize=4 才是运行时容量。默认 maxsize=0 反而代表无限容量。
    queue: asyncio.Queue[Record] = asyncio.Queue(maxsize=QUEUE_CAPACITY)
    # 空批次没有查询，仍给探针一个不会被使用的最小正目标；非空批次目标不超过 worker 或记录数。
    expected_peak: int = max(1, min(WORKER_COUNT, len(records)))
    # 条件表达式“已有对象 if 非空 else 新建对象”支持测试注入，并避免把可变实例写成默认参数。
    # 判断必须用 is not None：对象是否存在是身份语义，不应依赖对象未来可能定义的真假规则。
    run_probe: ConcurrencyProbe = probe if probe is not None else ConcurrencyProbe(expected_peak)
    run_metrics: QueueMetrics = metrics if metrics is not None else QueueMetrics()
    # 外层列表按 worker 编号保存结果：内层 [] 在每轮重新创建，二个 ... 工个共享同一个列表
```

```
408  async def run_worker_queue(
424      run_probe: ConcurrencyProbe = probe if probe is not None else ConcurrencyProbe(expected_peak)
425      run_metrics: QueueMetrics = metrics if metrics is not None else QueueMetrics()
426      # 外层列表按 worker 编号保存结果；内层 [] 在每轮重新创建，三个 worker 不会共享同一个列表。
427      per_worker_results: list[list[QueryResult]] = [[] for _ in range(WORKER_COUNT)]
428
429      # TaskGroup 同时拥有生产/协调者与全部 worker。任一成员抛非取消异常时，它会取消其余成员，
430      # 等所有成员的 finally 执行完，再向本函数调用者抛出 ExceptionGroup。
431      async with asyncio.TaskGroup() as group:
432          # 只有 1 个生产/协调 Task 和固定数量 worker，不随 records 数量增长。
433          # 这里不保存生产者句柄，因为生命周期和异常都由 TaskGroup 管理，不需要单独读取其结果。
434          group.create_task(
435              produce_then_join(
436                  queue,
437                  records,
438                  run_metrics,
439                  producer_fail_after=producer_fail_after,
440              ),
441              name="producer-and-joiner",
442          )
443          # enumerate 把（索引，当前内层列表）解包到两个变量；每个 worker 获得不同的 own_results。
444          for worker_id, own_results in enumerate(per_worker_results):
445              group.create_task(
446                  queue_worker(
447                      worker_id,
448                      queue,
449                      own_results,
450                      run_probe,
451                      run_metrics,
452                      business_failure_id=business_failure_id,
453                      crash_id=crash_id,
454                  ),
455                  name=f"queue-worker-{worker_id}",
456              )
457
458      # TaskGroup 成功退出后，不再有 worker 修改这些列表；父作用域此时取得聚合所有权。
459      # 双层生成器先依次展开每个 worker 的结果，再由 sorted 按 record_id 恢复输入顺序：
```

```
408  async def run_worker_queue(
458      # TaskGroup 成功退出后，不再有 worker 修改这些列表；父作用域此时取得聚合所有权。
459      # 双层生成器先依次展开每个 worker 的结果，再由 sorted 按 record_id 恢复输入顺序；
460      # lambda 是只含一个表达式的匿名函数，这里告诉 sorted 从每个 QueryResult 取哪个排序键。
461      results: list[QueryResult] = sorted(
462          (result for own_results in per_worker_results for result in own_results),
463          key=lambda result: result.record_id,
464      )
465      # 基础设施 Task 始终为 1 个生产者加固定 worker 数，不随 len(records) 增长。
466      infrastructure_tasks: Literal[4] = 1 + WORKER_COUNT
467      # 返回结果之外也交回探针和指标，调用者可在所有 Task 停止后检查完整生命周期。
468      return results, run_probe, run_metrics, infrastructure_tasks
469
470
471  async def demonstrate_worker_queue() -> None:
472      print_section("4. 有界 Queue + 固定 worker: 24 条批量查询与背压")
473      # 24 条输入远大于 4 个排队槽和 3 个 worker，足以稳定触发生产者背压。
474      records: list[Record] = make_records(24)
475      # record_id=7 的预期失败会变成 QueryResult.error，不会让 TaskGroup 失败。
476      results: list[QueryResult], probe: ConcurrencyProbe, metrics: Queu…, task_count = await run_worker_queue(
477          records,
478          business_failure_id=7,
479      )
480
481      # 断言分四组核对：结果完整性、在途上限、Queue 容量/背压、正常关闭与 worker 回收。
482      assert len(results) == len(records)
483      assert sum(result.ok for result in results) == 23
484      assert probe.max_active == WORKER_COUNT
485      assert probe.active == 0
486      assert metrics.max_queued <= QUEUE_CAPACITY
487      assert metrics.backpressure_waits > 0
488      assert metrics.workers_started == metrics.workers_exited == WORKER_COUNT
489      assert metrics.graceful_shutdowns == 1
490      assert metrics.immediate_shutdowns == 0
491      print(f"  输入={len(records)}，长期存在的基础设施 Task={task_count}")
492      print(f"  Queue maxsize={QUEUE_CAPACITY}，观察到的最大排队={metrics.max_queued}")
```

```
471 async def demonstrate_worker_queue() -> None:
481     # 断言分四组核对：结果完整性、在途上限、Queue 容量/背压、正常关闭与 worker 回收。
482     assert len(results) == len(records)
483     assert sum(result.ok for result in results) == 23
484     assert probe.max_active == WORKER_COUNT
485     assert probe.active == 0
486     assert metrics.max_queued <= QUEUE_CAPACITY
487     assert metrics.backpressure_waits > 0
488     assert metrics.workers_started == metrics.workers_exited == WORKER_COUNT
489     assert metrics.graceful_shutdowns == 1
490     assert metrics.immediate_shutdowns == 0
491     print(f"  输入={len(records)}，长期存在的基础设施 Task={task_count}")
492     print(f"  Queue maxsize={QUEUE_CAPACITY}，观察到的最大排队={metrics.max_queued}")
493     print(f"  生产者因满队列等待的次数={metrics.backpressure_waits}")
494     print(f"  worker={WORKER_COUNT}，观察到的最大在途={probe.max_active}")
495     print("  业务结果：成功 23，失败 1；三个 worker 均正常退出")
496
497
498 async def demonstrate_empty_batch() -> None:
499     print_section("5A. 空批次：关闭空 Queue，所有 worker 都会退出")
500     # 空列表仍启动固定基础设施，但生产者立即 shutdown；下划线表示不使用返回的 Task 数。
501     results: list[QueryResult], probe: ConcurrencyProbe, metrics: Queu…, _ = await run_worker_queue([])
502     # 没有任何 Record 进入 query，三个阻塞在 get() 的 worker 仍应被正常关闭并退出。
503     assert results == []
504     assert probe.started_count == 0
505     assert metrics.workers_exited == WORKER_COUNT
506     print("  结果为空，实际查询次数=0，退出 worker 数=3")
507
508
509 async def demonstrate_worker_crash() -> None:
510     print_section("5B. worker 意外异常：TaskGroup 取消同伴，紧急关闭 Queue")
511     # 显式创建并注入状态对象，即使 run_worker_queue 抛异常，调用者仍可在 except* 后检查清理结果。
512     metrics = QueueMetrics()
513     probe = ConcurrencyProbe(WORKER_COUNT)
514     caught = 0
515     try:
```

```python

async def demonstrate_worker_crash() -> None:
    print_section("5B. worker 意外异常: TaskGroup 取消同伴，紧急关闭 Queue")
    # 显式创建并注入状态对象，即使 run_worker_queue 抛异常，调用者仍可在 except* 后检查清理结果。
    metrics = QueueMetrics()
    probe = ConcurrencyProbe(WORKER_COUNT)
    caught = 0
    try:
        await run_worker_queue(
            make_records(12),
            crash_id=1,
            probe=probe,
            metrics=metrics,
        )
    # TaskGroup 以 ExceptionGroup 抛出成员失败；except* 只匹配其中 WorkerCrashed 类型的叶子异常。
    # group.exceptions 是本次匹配到的子异常序列，这里预期只有 crash_id=1 产生的一项。
    except* WorkerCrashed as group:
        caught: int = len(group.exceptions)

    # active 回到 0 证明在途 query 都跑过 finally；worker 退出数和紧急关闭数证明基础设施已清理。
    assert caught == 1
    assert probe.active == 0
    assert metrics.workers_exited == WORKER_COUNT
    assert metrics.immediate_shutdowns == 1
    print("  捕获 WorkerCrashed 叶子数:", caught)
    print("  取消清理后 active=0, 紧急关闭次数=1")


async def demonstrate_producer_crash() -> None:
    print_section("5C. 生产者异常：不等待哨兵或 join，整体有限时退出")
    # 与 worker 崩溃相反，这次错误来自生产者；TaskGroup 会取消仍在 get/query 中的 worker。
    metrics = QueueMetrics()
    probe = ConcurrencyProbe(WORKER_COUNT)
    caught = 0
    try:
        await run_worker_queue(
```

```
536 async def demonstrate_producer_crash() -> None:
542     try:
543         await run_worker_queue(
544             make_records(12),
545             producer_fail_after=4,
546             probe=probe,
547             metrics=metrics,
548         )
549     # except* 从异常组提取生产者故障；未匹配的异常若存在仍会继续向外传播。
550     except* ProducerCrashed as group:
551         caught: int = len(group.exceptions)
552
553     # 不完整输入不能形成成功结果；这里只验证异常传播和所有清理路径已经收束。
554     assert caught == 1
555     assert probe.active == 0
556     assert metrics.workers_exited == WORKER_COUNT
557     assert metrics.immediate_shutdowns == 1
558     print("  捕获 ProducerCrashed 叶子数:", caught)
559     print("  未要求生产者在失败后继续投递哨兵，所有 worker 已回收")
560
561
562 async def demonstrate_user_cancellation() -> None:
563     print_section("5D. 用户取消：取消父任务，清理 worker 和在途调用")
564     # hold_queries 永远不 set：三个 worker 达到峰值后会统一停在第二道 Event，等待外部取消。
565     hold_queries = asyncio.Event()
566     # 第二个位置参数绑定 hold_after_peak；expected_peak=3 确保取消前恰好有三个在途查询。
567     probe = ConcurrencyProbe(WORKER_COUNT, hold_after_peak=hold_queries)
568     metrics = QueueMetrics()
569     # 把整个 worker 池包装成一个“父 Task”，模拟调用方取消一次请求。
570     # Python 3.14 的 eager_start=False 明确让新 Task 先进入事件循环待调度，而非在 create_task
571     # 调用中立刻执行 coroutine；后面的 Event 同样保证实验不依赖具体调度顺序。
572     parent: Task[tuple[list[QueryResult], Concurre... = asyncio.create_task(
573         run_worker_queue(
574             make_records(12),
575             probe=probe,
576             metrics=metrics,
```

```python
async def demonstrate_user_cancellation() -> None:
    parent: Task[tuple[list[QueryResult], Concurre... = asyncio.create_task(
        run_worker_queue(
            make_records(12),
            probe=probe,
            metrics=metrics,
        ),
        name="cancelled-queue-request",
        eager_start=False,
    )

    # 等三个 query 都已登记 active 后再取消，才能验证“持有在途资源时取消”的清理路径。
    await probe.peak_reached.wait()
    parent.cancel()
    try:
        # cancel() 不等于已经取消完成；await parent 会等待其 TaskGroup 取消 worker 并运行所有 finally。
        await parent
    except asyncio.CancelledError:
        # 记录调用方确实观察到取消。这里捕获是为了继续做断言，而不是把请求伪装成成功结果。
        cancelled_propagated = True
    else:
        raise AssertionError("用户取消不应被转换成一次成功的空结果")

    # cancelled() 核对最终 Task 状态；其余计数证明三个在途 query 和三个 worker 都已完成清理。
    assert parent.cancelled()
    assert cancelled_propagated
    assert probe.active == 0
    assert probe.cleaned_count == WORKER_COUNT
    assert metrics.workers_exited == WORKER_COUNT
    assert metrics.immediate_shutdowns == 1
    print("  调用方仍收到 CancelledError:", cancelled_propagated)
    print("  在途清理数=3, 退出 worker 数=3, active=0")


async def one_item_worker(
    queue: asyncio.Queue[str],
```

```python
async def one_item_worker(
    queue: asyncio.Queue[str],
    started: asyncio.Event,
    processed: asyncio.Event,
) -> None:
    # 这个最小 worker 专门演示：处理一项并正确 task_done 后，循环仍会回到下一次 get() 等待。
    started.set()
    while True:
        try:
            item: str = await queue.get()
        except asyncio.QueueShutDown:
            # shutdown 才是生命周期退出信号；join() 返回不会自动走到这个分支。
            return
        try:
            # assert 固定本实验只处理预期字符串；set() 通知父任务业务代码已经执行。
            assert item == "work"
            processed.set()
        finally:
            # 只要 get 成功，每项都在 finally 中恰好记账一次。
            queue.task_done()


async def demonstrate_queue_accounting_bugs() -> None:
    print_section("6. task_done/join 记账，以及 join 不负责关闭 worker")

    # 不传 maxsize 等价于 maxsize=0，表示无界 Queue；此处只放一项，容量不是实验重点。
    forgotten: asyncio.Queue[str] = asyncio.Queue()
    # put 让 Queue 的未完成计数加 1；get 只取走项目，并不会把这个计数减回 0。
    await forgotten.put("work")
    assert await forgotten.get() == "work"
    async with asyncio.TaskGroup() as group:
        # join coroutine 被包装为 Task，父任务才能在它等待期间检查 done() 状态并补上遗漏记账。
        stuck_join: Task[None] = group.create_task(forgotten.join(), name="stuck-join-demo")
        # 显式让 join Task 运行到等待点；不是等待某个毫秒数或猜测调度先后。
        await asyncio.sleep(0)
        assert not stuck_join.done()
```

```
627  async def demonstrate_queue_accounting_bugs() -> None:
635      async with asyncio.TaskGroup() as group:
636          # join coroutine 被包装为 Task，父任务才能在它等待期间检查 done() 状态并补上遗漏记账。
637          stuck_join: Task[None] = group.create_task(forgotten.join(), name="stuck-join-demo")
638          # 显式让 join Task 运行到等待点；不是等待某个毫秒数或猜测调度先后。
639          await asyncio.sleep(0)
640          assert not stuck_join.done()
641          print("  教学反例: get 后漏掉 task_done, join 仍在等待:", not stuck_join.done())
642          # 这次 task_done 与前面的唯一一次 get 配对，未完成计数归零后 stuck_join 会被唤醒。
643          forgotten.task_done()  # 修复记账，让受控反例可以继续退出。
644      # TaskGroup 退出意味着它已等待 stuck_join 真正结束，不会把教学反例泄漏给下一段。
645      assert stuck_join.done()
646
647      # 第二个实验的 Queue 记账完全正确，专门证明“记账结束”和“worker 生命周期结束”不是一回事。
648      queue: asyncio.Queue[str] = asyncio.Queue()
649      worker_started = asyncio.Event()
650      processed = asyncio.Event()
651      async with asyncio.TaskGroup() as group:
652          worker: Task[None] = group.create_task(
653              one_item_worker(queue, worker_started, processed),
654              name="one-item-worker",
655          )
656          # 先确认 worker 已启动，再投递唯一工作；processed 则确认 worker 已执行这一项。
657          await worker_started.wait()
658          await queue.put("work")
659          await processed.wait()
660          # 等对应 task_done 后 join 返回，但 worker 已循环到下一次 queue.get()，仍是 pending 状态。
661          await queue.join()
662          assert not worker.done()
663          print("  join 已返回，但等待下一项的 worker 仍未退出:", not worker.done())
664          # 正常关闭空 Queue 会使等待 get() 的 worker 收到 QueueShutDown 并 return。
665          queue.shutdown()
666
667      # TaskGroup 已等待 worker 的 return 完成，所以此处 done() 必须为 True。
668      assert worker.done()
669      print("  显式 shutdown 后 worker 已退出:", worker.done())
```

```python
async def racy_increment(
    counter: Counter,
    read_finished: asyncio.Event,
    may_write: asyncio.Event,
) -> None:
    # 先把共享字段复制到局部不可变 int；两个 Task 都可能读到同一个旧值 0。
    snapshot: int = counter.value
    # 每个 Task 设置自己的 Event，父任务据此知道两次读取都已发生。
    read_finished.set()
    # 两个 Task 都在读完旧值后停住，父任务再统一放行写入。
    await may_write.wait()
    # 两个局部 snapshot 都是 0，所以随后两次赋值都写 1，后写者覆盖前写者而丢失一次增量。
    counter.value = snapshot + 1


async def locked_increment(counter: Counter, lock: asyncio.Lock) -> None:
    # async with lock 会等待互斥所有权，并在正常、异常或取消退出时释放；同一时刻仅一个 Task 进入。
    async with lock:
        snapshot: int = counter.value
        # 仅为教学故意在读写之间让出控制权；Lock 阻止另一个 Task 进入。
        # 真实代码不应无故持锁等待慢 I/O；这里的 await 是为了证明锁覆盖了完整“读–写”不变量。
        await asyncio.sleep(0)
        counter.value = snapshot + 1


async def count_owned_items(items: list[int]) -> int:
    # 模拟异步工作后只返回本 Task 的局部结果；函数不读取或写入共享 Counter。
    await asyncio.sleep(0)
    return len(items)


async def demonstrate_shared_state() -> None:
    print_section("7. 单线程仍会逻辑竞争：Lock 或改变数据所有权")
    # 两个 Task 共享 counter，却各有一个“已读完”信号；may_write 是父任务统一放行写入的广播信号。
    counter = Counter()
    first_read = asyncio.Event()
```

```
702
703 async def demonstrate_shared_state() -> None:
704     print_section("7. 单线程仍会逻辑竞争：Lock 或改变数据所有权")
705     # 两个 Task 共享 counter，却各有一个“已读完”信号；may_write 是父任务统一放行写入的广播信号。
706     counter = Counter()
707     first_read = asyncio.Event()
708     second_read = asyncio.Event()
709     may_write = asyncio.Event()
710
711     async with asyncio.TaskGroup() as group:
712         # 两个 coroutine 参数里的 counter/may_write 指向同一对象，而 read_finished 各不相同。
713         group.create_task(racy_increment(counter, first_read, may_write))
714         group.create_task(racy_increment(counter, second_read, may_write))
715         # 等两个独立信号都 set，证明两个 snapshot 都已读完旧值；此时再同时放行写入。
716         await first_read.wait()
717         await second_read.wait()
718         may_write.set()
719
720     # TaskGroup 已等待两次写入结束；确定性结果 1 证明单线程事件循环仍有跨 await 的逻辑竞争。
721     assert counter.value == 1
722     print("  两次“读–await–写”后的错误结果:", counter.value)
723
724     # 修复一：两个 Task 仍共享对象，但也共享同一把 Lock，把读取和写回作为一个完整临界区。
725     protected = Counter()
726     lock = asyncio.Lock()
727     async with asyncio.TaskGroup() as group:
728         group.create_task(locked_increment(protected, lock))
729         group.create_task(locked_increment(protected, lock))
730     # 第二个 Task 必须在第一个写完 1 后才能读，所以最终稳定得到 2。
731     assert protected.value == 2
732     print("  Lock 保护同一不变量后的结果:", protected.value)
733
734     # 修复二通常更清楚：每个 Task 只算自己的值，父任务在作用域退出后合并，不再共享可变计数器。
735     async with asyncio.TaskGroup() as group:
736         left: Task[int] = group.create_task(count_owned_items([1, 2]))
737         right: Task[int] = group.create_task(count_owned_items([3, 4, 5]))
```

```python
async def demonstrate_shared_state() -> None:
    # 修复二通常更清楚：每个 Task 只算自己的值，父任务在作用域退出后合并，不再共享可变计数器。
    async with asyncio.TaskGroup() as group:
        left: Task[int] = group.create_task(count_owned_items([1, 2]))
        right: Task[int] = group.create_task(count_owned_items([3, 4, 5]))
    # 离开 TaskGroup 后两个 Task 都已成功完成，result() 取回 2 和 3；父任务是唯一聚合者。
    owned_total: int = left.result() + right.result()
    assert owned_total == 5
    print("  子任务拥有局部结果、父任务最终合并:", owned_total)


async def run_examples() -> None:
    # 同步函数直接调用；各 async 实验按顺序 await，因此上一段完成清理后才进入下一段。
    explain_independent_limits()
    await demonstrate_semaphore_cancellation()
    await demonstrate_small_semaphore_batch()
    await demonstrate_worker_queue()
    await demonstrate_empty_batch()
    await demonstrate_worker_crash()
    await demonstrate_producer_crash()
    await demonstrate_user_cancellation()
    await demonstrate_queue_accounting_bugs()
    await demonstrate_shared_state()
    # all_tasks() 返回当前事件循环中尚未结束的 Task 集合，其中必然包含正在执行本函数的 Task。
    # 推导式排除 current 后应为空，用作整章最后一道“没有遗留后台 Task”的断言。
    current: Task[Any] | None = asyncio.current_task()
    leaked: list[Task[Any]] = [task for task in asyncio.all_tasks() if task is not current]
    assert leaked == [], f"发现未回收 Task: {leaked}"
    print("\n所有第 23 章示例与运行时断言均已完成。")


def main() -> None:
    """只创建一个事件循环；每个示例离开前都回收自己创建的 Task。"""
    # asyncio.run 接受 run_examples() 产生的 coroutine，创建事件循环并运行到完成，最后关闭循环。
    # Java 可把它类比为应用最外层的异步运行入口，但这里的事件循环和 Task 由 asyncio 管理。
    asyncio.run(run_examples())
```

```
744  async def run_examples() -> None:
759      leaked: list[Task[Any]] = [task for task in asyncio.all_tasks() if task is not current]
760      assert leaked == [], f"发现未回收 Task: {leaked}"
761      print("\n所有第 23 章示例与运行时断言均已完成。")
762
763
764  def main() -> None:
765      """只创建一个事件循环；每个示例离开前都回收自己创建的 Task。"""
766      # asyncio.run 接受 run_examples() 产生的 coroutine，创建事件循环并运行到完成，最后关闭循环。
767      # Java 可把它类比为应用最外层的异步运行入口，但这里的事件循环和 Task 由 asyncio 管理。
768      asyncio.run(run_examples())
769
770
771  # 模块被直接执行时 __name__ 为 "__main__"；被测试或别的模块 import 时只定义对象，不自动运行。
772  if __name__ == "__main__":
773      main()
774
```